40.3 ACP 控制面与集成模式

> **生成模型**：OpenAI GPT-5.4（ `openai/gpt-5.4` ）
> **Token 消耗**：输入 ~18k tokens，输出 ~3.5k tokens（本节）

如果说 32.1 讲的是 ACP（Agent Communication Protocol，Agent 通信协议）这门“语言”本身，那么这一节看的就是 OpenClaw 怎么把这门语言接进自己的控制面。这里的重点不再是协议字段，而是协议落地以后，谁来持有会话、谁来转发事件、谁来保证多轮对话不中断，以及外部渠道断线重连后，为什么还能接回原来的 ACP 会话。

40.3.1 控制面架构

OpenClaw 的 ACP 控制面核心是 `src/acp/control-plane/manager.core.ts` 里的 `AcpSessionManager` 。这个类是一个单例（singleton），原因并不复杂：ACP 运行时（runtime）句柄、活跃 turn、运行时缓存、错误统计，这些东西都不能分散到多个管理器实例里，否则同一个 `sessionKey` 可能被不同对象同时操作，状态马上就乱了。

`runTurn()` 是这套控制面的主路径。它的生命周期基本可以概括成六步：

1. 解析会话元数据，确认 `sessionKey` 对应的是一个可用的 ACP 会话。
2. 确保运行时句柄（runtime handle）存在；如果缓存里已有可复用句柄，就直接拿来，否则调用后端创建。
3. 把尚未下发的运行时控制项补齐，比如 mode、thinking、verbose 一类会话级选项。
4. 进入 `runtime.runTurn()` 的事件流，持续接收 `text_delta` 、 `tool_call` 、 `tool_call_update` 、 `status` 、 `done` 、 `error` 。
5. turn 结束后，如果不是一次性模式（oneshot），就做一次身份对账（identity reconcile），把运行时真实返回的会话标识重新写回元数据。
6. 如果是一次性模式，就直接关闭运行时并清理缓存。

源码里这条路径非常直白：

```
async runTurn(input: AcpRunTurnInput): Promise<void> {
  const resolution = this.resolveSession({ cfg: input.cfg, sessionKey });
  const resolvedMeta = requireReadySessionMeta(resolution);

  const { runtime, handle, meta } = await this.ensureRuntimeHandle({
    cfg: input.cfg,
    sessionKey,
    meta: resolvedMeta,
  });

  await this.applyRuntimeControls({ sessionKey, runtime, handle, meta });

  for await (const event of runtime.runTurn({
    handle,
    text: input.text,
    attachments: input.attachments,
    mode: input.mode,
    requestId: input.requestId,
    signal: combinedSignal,
  })) {
    await input.onEvent?.(event);
  }
}
```

这里有个很值得注意的分叉：**持久模式（persistent）** 和 **一次性模式（oneshot）**。两者的差别，不只是“会不会保留上下文”这么简单。

- 持久模式会把 `handle` 缓存在 `RuntimeCache` 里，下次同一会话再来，优先复用旧句柄。
- 持久模式在 turn 结束后会做身份对账，因为远端 ACP 后端有可能给这个会话重新分配真实的 `sessionId` 或其他标识。
- 一次性模式则不走缓存复用逻辑，turn 完成后立即 `runtime.close()`，理由写得很明确：`oneshot-complete`。

从控制面的角度看，`AcpSessionManager` 并不是“替代”运行时，而是在运行时上面又包了一层会话调度器（session scheduler）。它负责的是时序正确，而不是生成内容本身。

40.3.2 来源追踪（Provenance）

ACP 作为桥接层，有一个很现实的问题：当消息从 IDE、编辑器插件或者别的客户端流入 OpenClaw 时，后端怎么知道这条输入到底来自哪里？OpenClaw 在 `translator.ts` 里给了三档来源追踪

（Provenance）策略： `"off" | "meta" | "meta+receipt"` 。

第一档 `off` 最简单，不附加任何来源信息。这样做最“干净”，但后端拿到的就只是一段普通用户文本。

第二档 `meta` 会在系统输入里附加结构化元数据：

```
function buildSystemInputProvenance(originSessionId: string) {
  return {
    kind: "external_user",
    originSessionId,
    sourceChannel: "acp",
    sourceTool: "openclaw_acp",
  };
}
```

这里的 `InputProvenance` 可以理解成“简版来源标签”。它不改变用户原文，只是告诉后端：这段输入不是 Telegram、Discord 或 WebChat 直接送来的，而是 ACP 桥送来的外部用户消息。

第三档 `meta+receipt` 更进一步。除了结构化元数据，系统还会拼出一段 `[Source Receipt]` 文本块，内容包括：

- `bridge=openclaw-acp`
- `originHost`
- `originCwd`
- `acpSessionId`
- `targetSession`

对应实现大致如下：

```
function buildSystemProvenanceReceipt(params: {
  cwd: string;
  sessionId: string;
  sessionKey: string;
}) {
  return [
    "[Source Receipt]",
    "bridge=openclaw-acp",
    `originHost=${os.hostname()}`,
    `originCwd=${shortenHomePath(params.cwd)}`,
    `acpSessionId=${params.sessionId}`,
    `targetSession=${params.sessionKey}`,
    "[/Source Receipt]",
  ].join("\n");
}
```

这类 receipt 的价值，不在于“让模型多懂一点”，而在于给上游和下游之间补一条可审计的链路。尤其是调试多桥接场景时，开发者一眼就能看出来：这条消息是从哪台主机、哪个工作目录、哪个 ACP 会话过来的。

40.3.3 工具流式增强

ACP 客户端不满足于只看到“工具开始了”和“工具结束了”。如果中间什么都没有，前端体验会很僵。OpenClaw 在 `translator.ts` 里专门把工具事件拆成三个阶段： `start` 、 `update` 、 `result` 。

先看 `start` 。这一阶段的任务是把原始事件翻译成一个可展示的工具卡片：抽取工具名、参数、 `toolCallId` ，推断工具类型（kind），提取可能的文件位置（locations），再用 `formatToolTitle()` 生成标题。

```
if (phase === "start") {
  const args = data.args as Record<string, unknown> | undefined;
  const title = formatToolTitle(name, args);
  const kind = inferToolKind(name);
  const locations = extractToolCallLocations(args);
  await this.connection.sessionUpdate({
    sessionId: pending.sessionId,
    update: {
      sessionUpdate: "tool_call",
      toolCallId,
      title,
      status: "in_progress",
      rawInput: args,
      kind,
      locations,
    },
  });
}
```

再看 `update` 。这里传来的通常是 `partialResult` ，也就是工具的部分输出。翻译层会把它转成更适合 ACP 客户端消费的 `content` ，并且重新扫描一次位置集合，尽量把工具执行过程中暴露出来的路径也补进来。

`result` 阶段则是收口：拿最终结果、判断 `isError` ，然后把状态置成 `completed` 或 `failed` 。这一层做完以后，IDE 才知道某个工具卡片应该显示绿色完成，还是红色失败。

位置提取（location extraction）这一块也做得很细。 `event-mapper.ts` 不只是找 `path` 字段，它会同时识别多组路径键和行号键，还会扫描文本中的 `FILE:` 、 `MEDIA:` 标记。更重要的是，它给自己上了两道保险：最大递归深度 4、最大访问节点数 100。这就是典型的拒绝服务（DoS，Denial of Service）防护思路——外部事件数据再花，也不能让一个位置解析器无限下钻。

```
const TOOL_LOCATION_MAX_DEPTH = 4;
const TOOL_LOCATION_MAX_NODES = 100;

function collectToolLocations(value, locations, state, depth) {
  if (state.visited >= TOOL_LOCATION_MAX_NODES || depth > TOOL_LOCATION_MAX_DEPTH) {
    return;
  }
  // ...继续递归抽取 path / line / FILE / MEDIA 标记
}
```

去重策略也很实用：按 `path:line` 组合键去重。如果同一路径先抽到了“只有 path”，后面又抽到“path + line”，它会优先保留更精确的版本。对 IDE 来说，这直接决定了“点一下能不能跳到正确行”。

40.3.4 持久化绑定（Persistent Bindings）

ACP 还有一个很像“控制面工程”而不是“协议工程”的点：会话要能跟外部渠道绑定，而且这种绑定不能因为渠道断线、机器人重登、连接重建就丢掉。OpenClaw 这里做了持久化绑定（Persistent Bindings），目前主要覆盖 Discord 频道和 Telegram 话题（topic）。

相关代码拆成四个文件：

- `persistent-bindings.ts` ：统一导出入口。
- `persistent-bindings.route.ts` ：在路由阶段把外部会话映射到 ACP 绑定会话。
- `persistent-bindings.resolve.ts` ：从配置里解析绑定规则，生成规范化的 binding spec。
- `persistent-bindings.lifecycle.ts` ：负责确保会话存在、重配、原地 reset。

这套设计的关键不在“存了个映射表”，而在它把绑定会话本身做成了稳定的 `sessionKey` 。一旦绑定命中，路由层就会把当前消息改写到那个固定的目标会话上：

```
return {
  boundSessionKey,
  boundAgentId,
  route: {
    ...params.route,
    sessionKey: boundSessionKey,
    agentId: boundAgentId,
    matchedBy: "binding.channel",
  },
};
```

结果就是：哪怕 Discord 连接中断后又连上，或者 Telegram topic 被重新载入，只要渠道标识和配置绑定规则还对得上，消息就会继续落到原来的 ACP 会话里。对用户来说，这意味着“上下文没丢”；对实现者来说，这意味着控制面把“聊天渠道生命周期”和“ACP 会话生命周期”解耦了。

`persistent-bindings.lifecycle.ts` 里还有个很实在的处理：如果当前会话已经存在，但 agent、mode、backend、cwd 跟绑定配置不一致，就先关闭，再按绑定规格重新初始化。它不是赌旧状态还能凑合用，而是明确做一次重配置。

40.3.5 运行时控制与配置

OpenClaw 的 ACP 集成并不把会话看成一个“只能发 prompt 的黑箱”。相反，它把不少运行时控制项开放成了会话级配置，并持久化在 session metadata 里。

目前能看到的核心配置有五组：

- 思考强度（thought level）： `off` 到 `xhigh`
- 工具详细度（verbose）： `off` / `on` / `full`
- 推理流（reasoning）： `off` / `on` / `stream`
- 用量展示（usage）： `off` / `tokens` / `full`
- 提权级别（elevated）： `off` / `on` / `ask` / `full`

这些配置并不是前端随便发一个值，后端就照单全收。 `set_mode` 和 `set_config_option` 都要先经过 capability 校验。后端运行时如果没声明支持某个 control，管理器会直接返回 `ACP_BACKEND_UNSUPPORTED_CONTROL` 。

```
if (!capabilities.controls.includes("session/set_config_option") || !params.runtime.set
  throw createUnsupportedControlError({
    backend,
    control: "session/set_config_option",
  });
}
```

另外，控制项不是每个 turn 都盲目重放。 `applyManagerRuntimeControls()` 会先基于当前 runtime options 生成一个 signature，再和缓存中的 `appliedControlSignature` 比较。签名没变，就不重复下发。这其实是一个小优化，但效果很明显：减少后端控制调用，避免每轮对话都做一遍相同的会话配置。

更关键的是，这些配置会落进 session metadata，所以会话断开再恢复后，控制面可以把之前的 runtime options 再应用回去。换句话说，ACP 会话不只是“记住历史对话”，还会“记住行为方式”。

40.3.6 错误处理与安全

最后看错误处理和安全。ACP 这层代码的态度很明确：桥接层不能只是转发，还得做边界防御。

先看错误模型。OpenClaw 定义了 `AcpRuntimeError` ，用结构化错误码来表示后端缺失、后端不可用、控制项不支持、会话初始化失败、turn 失败等问题：

```
export const ACP_ERROR_CODES = [
  "ACP_BACKEND_MISSING",
  "ACP_BACKEND_UNAVAILABLE",
  "ACP_BACKEND_UNSUPPORTED_CONTROL",
  "ACP_DISPATCH_DISABLED",
  "ACP_INVALID_RUNTIME_OPTION",
  "ACP_SESSION_INIT_FAILED",
  "ACP_TURN_FAILED",
] as const;
```

这类设计的好处是，上层不用靠字符串猜错误语义。至于“能不能重试”，运行时事件层在 `error` 事件里还预留了 `retryable?: boolean` 标记，虽然 `AcpRuntimeError` 本身更偏向统一归一化，但控制面和客户端已经有了继续扩展的接口。

安全边界主要有三处。

第一处是会话创建限流（rate limiting）。 `translator.ts` 用固定窗口限流器约束 `newSession` 和 `loadSession` ，默认是 10 秒最多 120 次。这个数字不算特别小，但足够挡住明显异常的批量打点请求。

第二处是提示长度上限。 `MAX_PROMPT_BYTES` 被设成 2MB，并且不是等全文拼完再检查，而是在 `extractTextFromPrompt()` 逐块累加字节数时就提前拒绝。这里指向的是典型的资源耗尽问题，即 CWE-400。

```
const MAX_PROMPT_BYTES = 2 * 1024 * 1024;

if (totalBytes > maxBytes) {
  throw new Error(`Prompt exceeds maximum allowed size of ${maxBytes} bytes`);
}
```

第三处是桥接模式的能力收缩。ACP bridge 明确拒绝每会话 MCP 服务器（per-session MCP servers）：

```
private assertSupportedSessionSetup(mcpServers: ReadonlyArray<unknown>): void {
  if (mcpServers.length === 0) return;
  throw new Error(
    "ACP bridge mode does not support per-session MCP servers. Configure MCP on the Ope
  );
}
```

这背后的考虑很现实：如果桥接层允许每个 ACP 会话动态挂载一组 MCP（Model Context Protocol，模型上下文协议）服务，OpenClaw 的控制面就要额外承担会话级外部能力注入的安全风险。当前实现选择把这件事压回 Gateway 或 agent 侧统一配置，边界更清楚，也更容易审计。

本节小结

ACP 在 OpenClaw 里并不是一层薄薄的协议适配，而是一套完整的控制面实现。 `AcpSessionManager` 负责会话解析、运行时句柄缓存、turn 生命周期调度，以及 persistent/oneshot 两种模式下不同的收尾策略； `translator.ts` 则把 Gateway 事件翻译成更适合 ACP 客户端消费的会话更新，并补上 provenance、工具流式增强和安全校验。再往外一层，持久化绑定把 Discord 频道、Telegram 话题这类外部对话入口，稳定地锚定到 ACP 会话上。这样一来，ACP 不只是“能接进来”，而是真的被纳入了 OpenClaw 的长期会话控制体系。